3. Linear Models for Classification

Gholamreza Haffari, Borhan Kazimipour

3. Linear Models for Classification

Gholamreza Haffari, Borhan Kazimipour

Generated by Alexandria (https://www.alexandriarepository.org) on May 22, 2017 at 10:51 pm AEST

Contents

Title i
Copyright ii
1 Basic Concepts 1
2 Discriminative Models 6
3 Activity 3.1 Perceptron 13
4 Probabilistic Generative Models 14
5 Activity 3.2 Bayesian Classifier 18
6 Probabilistic Discriminative Models 19
7 Activity 3.3 Logistic Regression 21

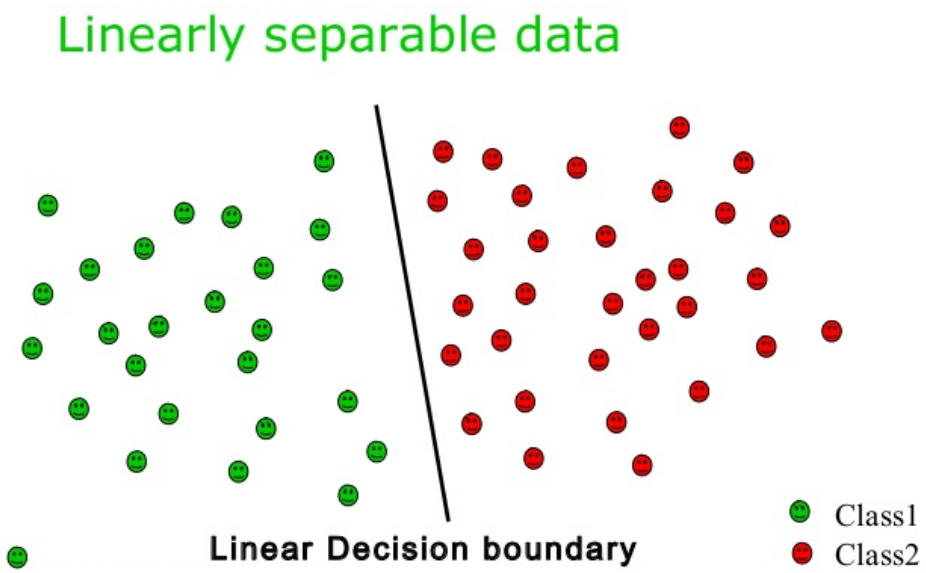
1
Basic Concepts

In [Module 2](https://www.alexandriarepository.org/?post_type=syllabus&p=84662) (https://www.alexandriarepository.org/?post_type=syllabus&p=84662), we explored *linear models* for *regression,* which result in simple training algorithms and prediction rules. We now present linear models for *classification.* The goal in classification tasks is to take an observation vector $\mathbf{x}$ and to assign it to one of K *discrete* classes $\mathcal{C}_k$ where $k = 1, \ldots, K$. Note that, the main difference between classification and regression tasks is the type of *output* variable, which is discrete for classification but continues for regression.

The Loan Example

Suppose we would like to build a classifier which, given a *loan application*, predicts whether the it should be *rejected* or *accepted*. The classifier makes the prediction based on some *features* of the application, e.g. the salary of the applicant, the requested amount, etc.

In this example, each input data point is represented by a vector $\mathbf{x} \in \mathbb{R}^2$ consisting of two real-valued numbers. So each loan application can be represented as a point in a two-dimensional space where the horizontal axis corresponds to the applicant's salary, and the vertical axis corresponds to the requested loan amount (see Figure **3.1.1**). The classifier is trained on the training data consisting of some loans annotated with the reject (Class 1) or accept (Class 2) decision produced by a human expert. The class labels are colour coded by red and green for the training data points in Figure **3.1.1**. Once the classifier is trained, it predicts either reject (Class 1) or accept (Class 2) upon receiving the vector representation of a new loan application.



(http://www.slideshare.net/ANITALOKITA/winnow-vs-perceptron)

Figure 3.1.1 A Linearly Separable Binary Classification Problem. The black solid line indicates a linear decision boundary. Source: http://www.slideshare.net/ANITALOKITA/winnow-vs-perceptron

Linearly-Separable Problems

Input Space. Each input data point is represented by a vector in the *input space*.

Decision Boundary. Based on the class labels, the input space is divided into *decision regions* where each region corresponds to a class label. The decision regions are separated by the *decision boundaries* (e.g., the black solid line in Figure **3.1.1**).

In this module, we only consider linear models for classification, by which we mean that the decision boundaries are *linear functions* of the input vector. Therefore, if we have a 2-dimensional input space, the decision boundary will be a line. More generally, if the input space is *D*-dimensional, the decision boundary will be a (*D-1*)-dimensional *hyperplane.*

Linear Separability. Datasets whose classes can be separated by linear decision surfaces are said to be *linearly-separable* problems.

Classification Approaches

Generally speaking, there are three approaches to solving a classification problem, which we introduce in the following.

Discriminative Models. These are *non-probabilistic* models which directly assign the input $\mathbf{x}$ to a specific class $\mathcal{C}_k$. They are called discriminative models since they discriminate the classes for a given input. The simplest method to build a discriminant function is the Perceptron Algorithm, which we discuss in the following chapter.

Probabilistic Discriminative Models. Instead of assigning the input to a specific class, these models produce the *probability* of belonging to each class for a given input $P_{\boldsymbol{w}}(\mathcal{C}_k|\mathbf{x})$, where $\boldsymbol{w}$ is the model parameter and needs to be learned from the training data. The most common probabilistic discriminative model is the *Logistic Regression Model*, that we will see in chapter.

Probabilistic Generative Models. In these models, we are *not directly* modelling the class posterior $P_{\boldsymbol{w}}(\mathcal{C}_k|\mathbf{x})$. Instead, we learn the *class-prior distribution* and the *class-conditional distribution* and combine them based on the Bayes rule to get the class posterior:

$$P(\mathcal{C}_k|\mathbf{x}) \propto \underbrace{P_{\boldsymbol{w}}(\mathcal{C}_k)}_{\text{Class Prior}} \times \underbrace{P_{\boldsymbol{w}}(\mathbf{x}|\mathcal{C}_k)}_{\text{Class Conditional Density}}$$

where $\boldsymbol{w}$ is our model parameter vector.

Generative vs Discriminative Models. You may wonder why we may be interested to build our probabilistic classifier *indirectly* based on the generative model, versus the discriminative approach which *directly* models the class posterior. A simple answer is that we can generate synthetic data $\boldsymbol{x}$ if we have a generative model, whilst we cannot generate such data if we only have a discriminative model.

The power of generative models in generating synthetic data comes at a cost: Generative models often have *more parameters* compared to their corresponding discriminative models. This is intuitively due to the fact that generative models need to have some parameters to model the density of the input data, as opposed to discriminative models which do not need to model the input density. More parameters lead to more complexity, hence makes the model prone to various problems, e.g. overfitting.

Generalized Linear Models

Before going to the details of the linear models for classification, let's take a step back and look at the big picture. In particular, we want to find the link between linear models for regression that were covered in the previous module and the linear models for classification that we will present in this module. We provide a unified framework to see linear models for regression and classification from the same lens. This has the benefit that we may be able to re-use the regression learning algorithms for learning the classification models.

Recall that in the linear regression models, the model prediction $y(\mathbf{x})$ was given by a linear function of the parameters $\boldsymbol{w}$:

$$\text{Linear Regression Model:} \quad y(\mathbf{x}) = \boldsymbol{w}^T \mathbf{x} + w_0$$

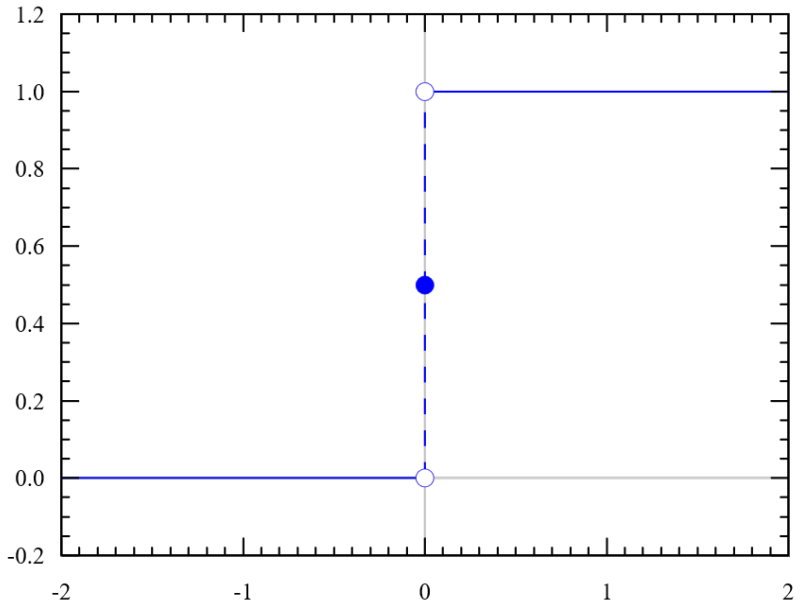
The output in regression models is a continuous, whereas in classification we need discrete values. To be able to have *discrete* output needed for classification problems, we generalise the linear regression model by applying a nonlinear function f to the regression output:

$$\text{Generalised Linear Model:} \quad y(\mathbf{x}) = f(\boldsymbol{w}^T \mathbf{x} + w_0)$$

Step Function. If we define f to be the *step function*, we get back the discrete values that we need:

$$f(u) = \begin{cases} 1 & \text{if } u \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

As we will see later, using this step function results in the *Perceptron Algorithm* to train a *discriminative model*. Note that although f is a nonlinear function, the decision boundary between the classes is going to be linear (see Chapter 2 for more geometrical insights).



(https://en.wikipedia.org/wiki/Step_function)

Figure 3.1.2 Plot of Step fnction Source: https://en.wikipedia.org/wiki/Step_function

Logistic Function. Alternatively, we can define f to be the *sigmoid* (a.k.a. *logistic*) function:

$$f(u) = \frac{1}{1+\exp(-u)}$$

The logistic function maps real values $u \in [-\infty, +\infty]$ to the range $f(u) \in [0, 1]$, which allows us to interpret the output as a *probability* of belonging to a class. We will see that this choice of the activation function results in a classification model called Logistic Regression, which is a *probabilistic discriminative model* (Figure **3.1.3**).

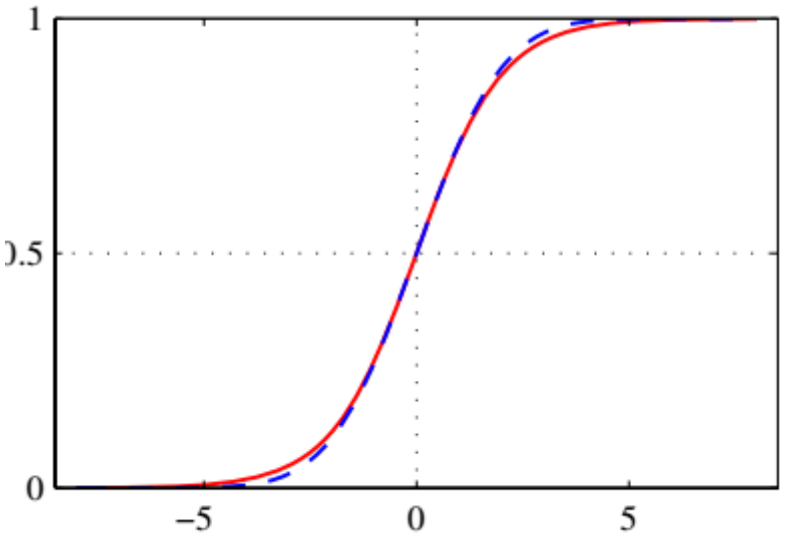


Figure 3.1.3 Plot of the logistic sigmoid functio

2
Discriminative Models

A *discriminant* is a function that takes an input vector $\mathbf{x}$ and assigns it to one of K classes. In this chapter we see the decision boundary induced by linear discriminants (for binary and multiple classification problems). We then see a powerful yet simple method, called the Perceptron Algorithm, to learn linear discriminants.

The Geometry of Linear Discriminants

Two Classes

The Model and Prediction Rule. A linear function of the input vector is the simplest linear discriminant: $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + w_0$ where $\mathbf{w}$ is a *weight vector*, and w_0 is a *bias* term. In a binary classification problem, an input vector $\mathbf{x}$ is assigned to class $\mathcal{C}_1$ if $y(\mathbf{x}) \geq 0$ and to class $\mathcal{C}_2$ otherwise. Recall that it is a generalised linear model with Step Function as the activation function.

Decision Boundary. The induced decision boundary by our model is linear. The reason is that for $\mathbf{x}$ to belong to $\mathcal{C}_1$S we should have $f(\mathbf{w}^T\mathbf{x} + w_0) \geq 0$ which happens iff $\mathbf{w}^T\mathbf{x} + w_0 \geq 0$. This means the line $\mathbf{w}^T\mathbf{x} + w_0 = 0$ separates the two classes, hence the induced decision boundary is linear (see Figure **3.2.1).** The decision surface, shown in red, is perpendicular to $\mathbf{w}$, and its displacement from the origin is controlled by the bias parameter w_0.

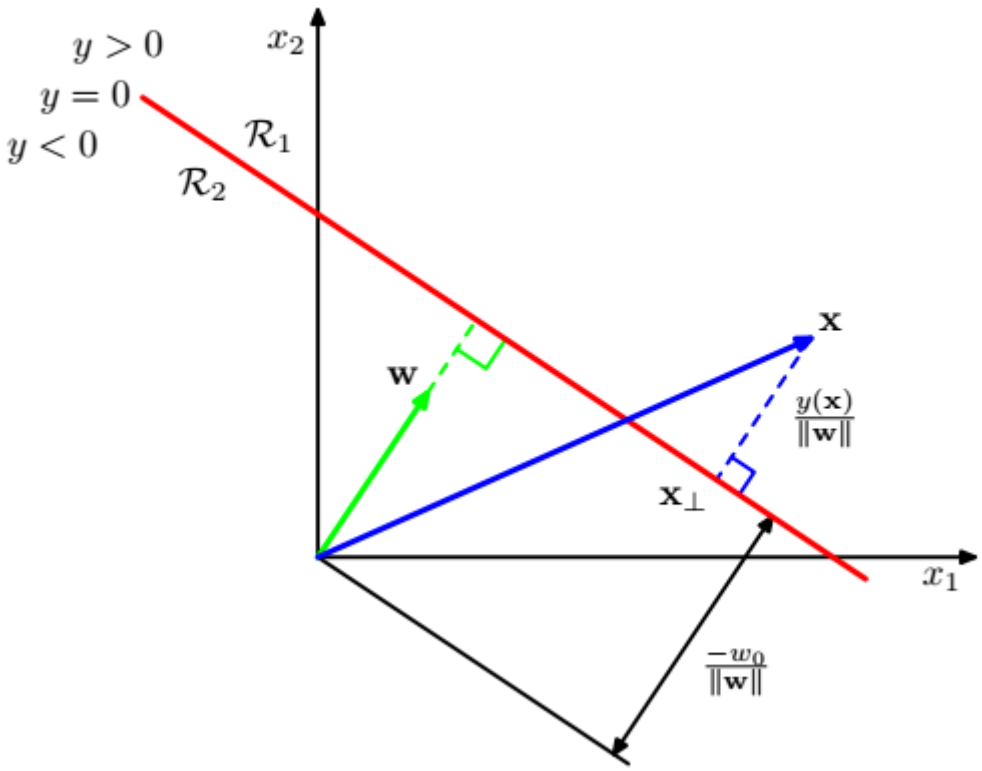


Figure 3.2.1. Illustration of the geometry of a linear discriminant function in two dimensions.

Multiple Classes

We start by two ideas to extend linear discriminants from binary to multiple classification. We show that these ideas have deficiency, and then present our main approach for the extension.

Initial Attempts. Our first idea, called *one-versus-rest*, is to use $K-1$ classifiers each of which solves a two-class problem of separating points in a particular class $\mathcal{C}_k$ from points not in that class (Figure **3.2.2**, left**).** The second idea, called *one-versus-one*, is to build $K(K-1)/2$ binary discriminant functions, one for every possible pair of classes. Each point is then classified according to a majority vote among the discriminant functions (Figure **3.2.2**, right). The main issue with these ideas is that both of them result in *ambiguous regions* (marked in green in Figure **3.2.2**) belonging to none of the classes.

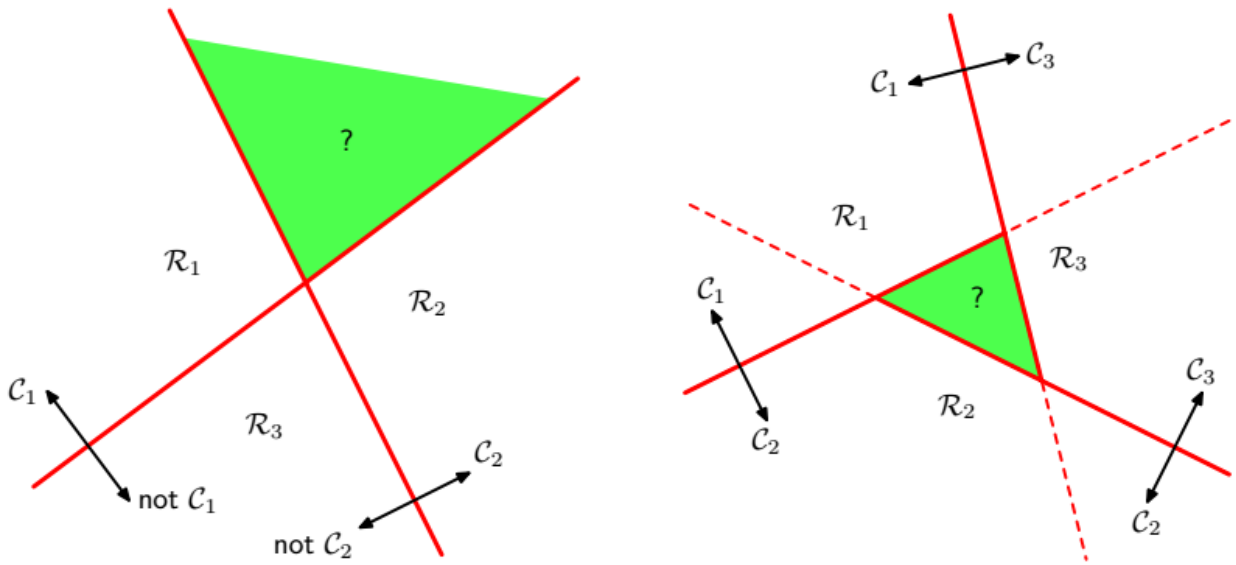


Figure 3.2.2. Attempting to construct a K-class discriminant from a set of two class discriminants leads to ambiguous regions, shown in green, based on *one-versus-rest* (left) and *one-versus-one* (right) ideas.

The Model and Prediction Rule. Corresponding to each class k, we consider a linear functions of the form $y_k(\mathbf{x}) = \mathbf{w}_k^T\mathbf{x} + w_{k0}$. We then assign a point $\mathbf{x}$ to a class $\mathcal{C}_k$ if $y_k(\mathbf{x}) > y_j(\mathbf{x})$ for all $j \neq k$.

Decision Boundary. The resulting decision regions of such a discriminant are always singly connected and linear, *without* any ambiguous region (see Figure **3.2.3**). The reason is that the decision boundary between class $\mathcal{C}_k$ and class $\mathcal{C}_j$ is given by $y_k(\mathbf{x}) = y_j(\mathbf{x})$ and hence corresponds to a (D − 1)-dimensional *hyperplane* defined by $(\mathbf{w}_k - \mathbf{w}_j)^T\mathbf{x} + (w_{k0} - w_{j0}) = 0$. This has the same form as the decision boundary for the two-class case and so analogous geometrical properties apply.

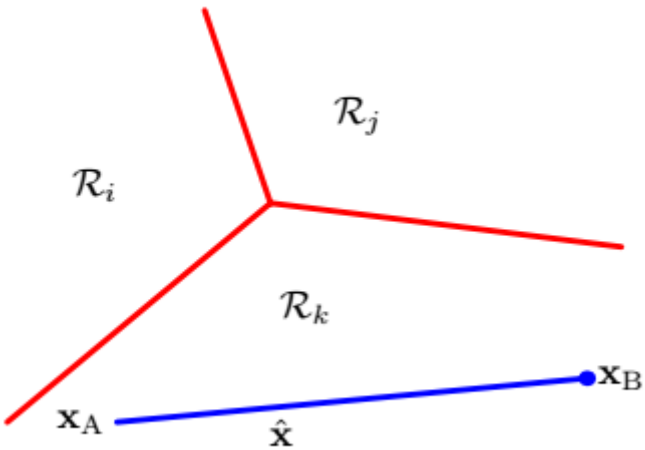


Figure 3.2.3.Illustration of the decision regions for a multiclass linear discriminant. The red lines represent decision boundaries.

The Perceptron

The Perceptron, which is developed by Rosenblatt (https://en.wikipedia.org/wiki/Frank_Rosenblatt) in the 1950s, is a classic example of a discriminative model with a very intuitive learning algorithm.

The Model and Prediction Rule. As we saw in Chapter 1, Perceptron is simply a generalised linear model where the activation function is the step function:

$$y(x) := f(\boldsymbol{w} \cdot \phi(\boldsymbol{x}))$$

where $\phi(.)$ can be generally a nonlinear transformation providing a new representation of the input (it can be the identity function providing $\boldsymbol{x}$ itself), and $f(.)$ is the *modified step function* providing the class prediction rule:

$$f(u) = \begin{cases} +1 & \text{if } u \geq 0 \\ -1, & \text{otherwise} \end{cases}$$

Note that we have represented the class $\mathcal{C}_1$ by the value $+1$ and the class $\mathcal{C}_2$ by the value -1.

The Training Objective. Suppose we are given a set of N training example $\{(\boldsymbol{x}_1, t_1), \ldots, (\boldsymbol{x}_N, t_N)\}$ consisting of input data points $\boldsymbol{x}_n$ annotated with their correct class label $t_n \in \{-1, +1\}$. The aim is to learn the parameters of the model $\boldsymbol{w}$. The basic idea in training the perceptron is to *minimise the number of misclassified training data points.*

A training data point is misclassified when $\boldsymbol{w} \cdot \phi(\boldsymbol{x}_n) t_n < 0$. For example, if $\boldsymbol{w} \cdot \phi(\boldsymbol{x}_n) > 0$ we know that the predicted label is going to be $+1$. Now if the true label t_n is -1, then an error has occurred. These conditions for identifying a misclassified training example has been captured beautifully by the following condition $\boldsymbol{w} \cdot \phi(\boldsymbol{x}_n) t_n < 0$.

As said, the training objective in perceptron is to minimise the number of misclassified examples:

$$E(\boldsymbol{w}) := -\sum_{n \in \mathcal{M}} \boldsymbol{w} \cdot \phi(\boldsymbol{x}_n) t_n$$

where $\mathcal{M}$ is just the *set of misclassified examples* (not all of the training data points). Ideally, we would like to find the parameter vector $\boldsymbol{w}$ such that the above *error* function is zero, i.e. a parameter vector which classifies all the training examples correctly.

The Perceptron Learning Algorithm. We can apply the stochastic gradient descent (SGD) algorithm from Module 2 (https://www.alexandriarepository.org/reader-preview/84662/91471) to minimise the error objective as a function of parameters $\boldsymbol{w}$.

The algorithm works as follows. Initially, the parameters are set to zero. Then after processing each data

point $(\boldsymbol{x}_n, t_n)$, *if the datapoint is currently misclassified* then the parameter vector is updated via:

$$\boldsymbol{w}^{(\tau+1)} := \boldsymbol{w}^{(\tau)} - \eta^{(\tau)} \nabla E(\boldsymbol{w}) = \boldsymbol{w}^{(\tau)} + \eta^{(\tau)} \phi(\boldsymbol{x}_n) t_n$$

where τ is the iteration number, and $\eta^{(\tau)}$ is the iteration-specific learning rate. Note that if the training data point is classified correctly, we do not make the above update and proceed to process the next training example. The algorithm stops when a stopping condition is met, for example the difference between the values of the objective function in two consecutive iterations is less than a threshold $|E(\boldsymbol{w}^{(\tau+1)}) - E(\boldsymbol{w}^{(\tau)})| < \epsilon$.

The perceptron learning algorithm has a simple interpretation, as follows. We cycle through the training patterns in turn, and for each pattern $\boldsymbol{x}_n$ we apply the perceptron prediction rule. If the pattern is correctly classified, then the weight vector remains unchanged, whereas if it is incorrectly classified, then for class $\mathcal{C}_1$ we add the vector $\phi(\boldsymbol{x}_n)$ onto the current estimate of weight vector $\boldsymbol{w}$ while for class $\mathcal{C}_2$ we subtract the vector $\phi(\boldsymbol{x}_n)$ from $\boldsymbol{w}$. The perceptron learning algorithm is illustrated in Figure **3.2.4.**

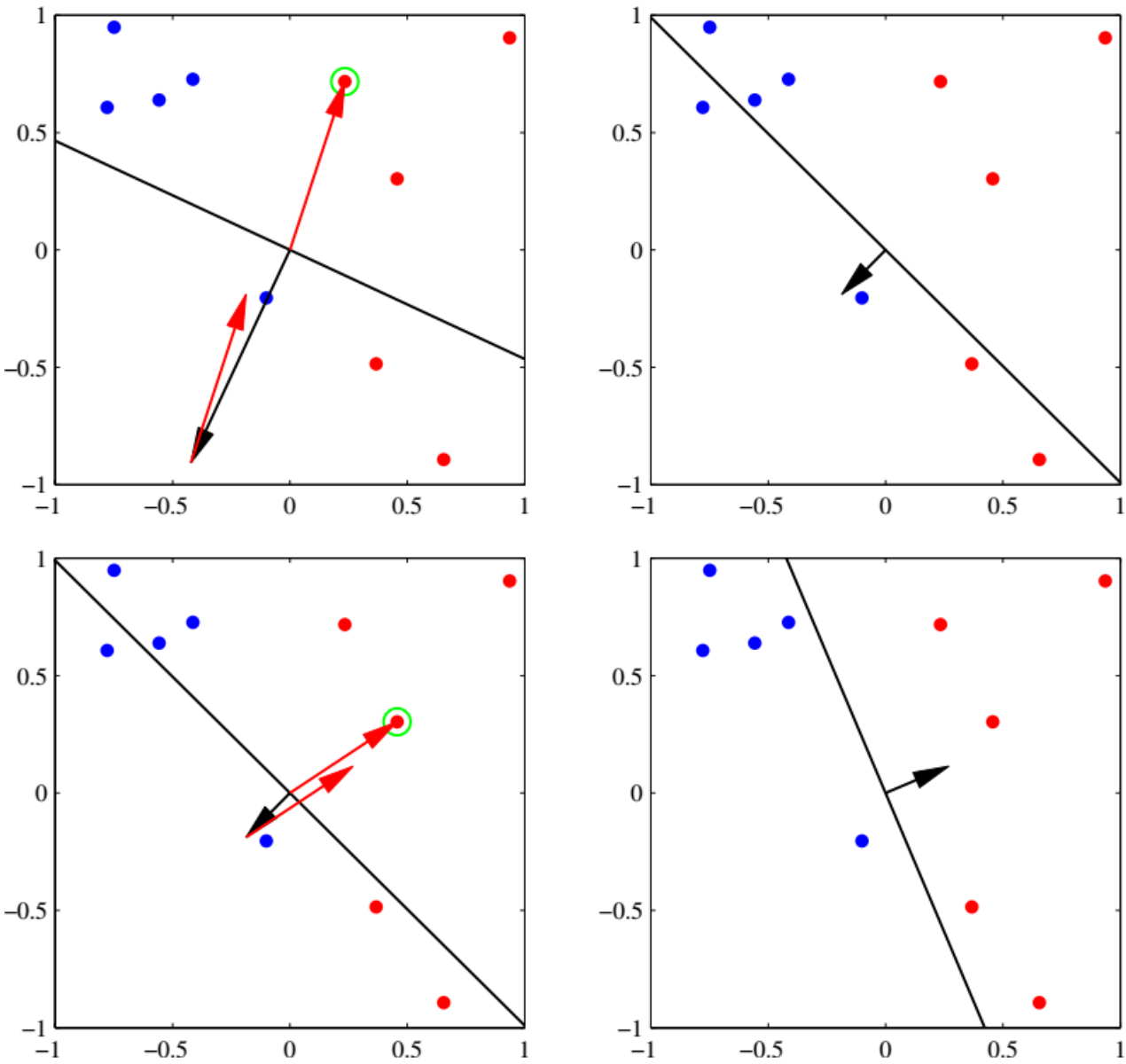


Figure 3.2.4. Illustration of the convergence of the perceptron learning algorithm, showing data points from two classes (red and blue) in a two-dimensional feature space (φ1, φ2). The top left plot shows the initial parameter vector w shown as a black arrow together with the corresponding decision boundary (black line), in which the arrow points towards the decision region which classified as belonging to the red class. The data point circled in green is misclassified and so its feature vector is added to the current weight vector, giving the new decision boundary shown in the top right plot. The bottom left plot shows the next misclassified point to be considered, indicated by the green circle, and its feature vector is again added to the weight vector giving the decision boundary shown in the bottom right plot for which all data points are correctly classified.

Important Remarks

1. If the data is separable, the perceptron training algorithm is guaranteed to find *a* perfect parameter vector (i.e. a parameter vector with error zero). However, the algorithm may need too many iterations over the dataset to converge to a perfect parameter vector.
2. The perceptron algorithm may not converge if the data is not separable. In this case, the algorithm may cycle through some parameter vectors which have the same value for the objective function, hence the algorithm visits them without stopping.
3. The perceptron algorithm is sensitive to initialisation. For example, in the case where the data is separable, there are many perfect parameter vectors with error zero. Two runs of the algorithm

may converge to different perfect parameters if they visit the training examples in the training set in different orders.

3
Activity 3.1 Perceptron

Perceptron

In this activity we implement a perceptron for two-class classification problems. This activity helps you to complete Assignment 2.

Please download this JuPyteR file (Activity.3.1 (https://www.alexandriarepository.org/wp-content/uploads/20160901075046/Activity.3.1.ipynb) right click and then choose "Save link as").

For detailed discussion about setting your programming environment up, please refer to Appendix B: Setting up Your Programming Environme (https://www.alexandriarepository.org/?post_type=module&p=97654)

4
Probabilistic Generative Models

Generative Models

In the previous chapter, we only looked at learning classification by choosing parameters to minimize an error function. Here we shall adopt a generative (probabilistic) approach in which we model the class-conditional densities $p(\mathbf{x}|\mathcal{C}_k)$, as well as the class priors $p(\mathcal{C}_k)$, and then use these to compute posterior probabilities $p(\mathcal{C}_k|\mathbf{x})$ through Bayes' theorem.

From Bayes' theorem, we know

$$p(\mathcal{C}_1|\mathbf{x}) = \frac{p_{\boldsymbol{\theta}}(\mathbf{x}|\mathcal{C}_1)p_{\boldsymbol{\theta}}(\mathcal{C}_1)}{\underbrace{p_{\boldsymbol{\theta}}(\mathbf{x}|\mathcal{C}_1)p_{\boldsymbol{\theta}}(\mathcal{C}_1)+p_{\boldsymbol{\theta}}(\mathbf{x}|\mathcal{C}_2)p_{\boldsymbol{\theta}}(\mathcal{C}_2)}_{p_{\boldsymbol{\theta}}(x)}}$$

The intuition is that we have assumed the following *hypothetical* process that has generated the training data point $(\mathbf{x}, \mathbf{t})$ given to us:

- First, the class label t has been generated, by tossing a coin, where classes $\mathcal{C}_1$ and $\mathcal{C}_2$ correspond to Heads and Tails respectively.
- Second, the input vector $\boldsymbol{x}$ has been generated, by sampling from the class conditional distribution corresponding to the label t.

The Geometry of Generative Classifiers

Let us assume the class-conditional densities are multivariate Gaussians and have the same covariance matrix. Our goal is to get insight about the prediction rule and show that the induced decision boundary is linear.

Class-Conditional Densities. The parameters of the model are the covariance matrix $\boldsymbol{\Sigma}$ common to the two classes, and class-specific mean vectors $\boldsymbol{\mu}_1$ and $\boldsymbol{\mu}_2$. Therefore, the class conditional density for the class $\mathcal{C}_k$ is as follows:

$$p_{\boldsymbol{\theta}}(x|\mathcal{C}_k) = \frac{1}{(2\pi)^{D/2}|\Sigma|^{1/2}} \exp\left[-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_k)^T\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_k)\right]$$

where D is the dimension of $\boldsymbol{x}$ (see Figure **3.4.1**, left).

Prediction Rule. We assign $\boldsymbol{x}$ to class $\mathcal{C}_1$ if its posterior probability is bigger than that for class $\mathcal{C}_2$. Using the Bayes rule to plug-in the posterior probabilities, our prediction rule can be re-written as:

$$p(\mathcal{C}_1|x) > p(\mathcal{C}_2|x) \Leftrightarrow \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x})} > \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x})}$$

Since the denominators are the same, we only need to check the following to predict the class $p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1) > p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)$.

Alternatively, let us introduce a as

$$a = \ln \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)}$$

The prediction rule becomes to predict class $\mathcal{C}_1$ if $a > 0$, and class $\mathcal{C}_2$ otherwise.

The Decision Boundary. Plugging the Gaussian class-conditional densities, we can get a simplified formulation for a as follows:

$$\begin{aligned} a &= \ln \frac{p(\mathbf{x}|\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_2)} + \ln \frac{p(\mathcal{C}_1)}{p(\mathcal{C}_2)} \\ &= \ln \frac{\exp\left[-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_1)^T\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_1)\right]}{\exp\left[-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_2)^T\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_2)\right]} + \ln \frac{p(\mathcal{C}_1)}{p(\mathcal{C}_2)} \\ &= \boldsymbol{w}\cdot\boldsymbol{x} + w_0 \end{aligned}$$

where we note that the quadratic term $\boldsymbol{x}^T\boldsymbol{\Sigma}^{-1}\boldsymbol{x}$ is cancelled. Hence a takes a simple linear form. This means that the induced decision boundary is linear.

Connection to Generalised Linear Models. Based on our definition of a, we can reformulate the posterior probability $p(\mathcal{C}_1|\mathbf{x})$ as:

$$p(\mathcal{C}_1|\mathbf{x}) = \frac{1}{1+e^{-a}} = \sigma(a) = \sigma(\boldsymbol{w}\cdot\boldsymbol{x} + w_0)$$

Hence, our probabilistic generative classifier results indeed in a generalised linear model, where the activation function is sigmoid (Figure **3.4.1**, right).

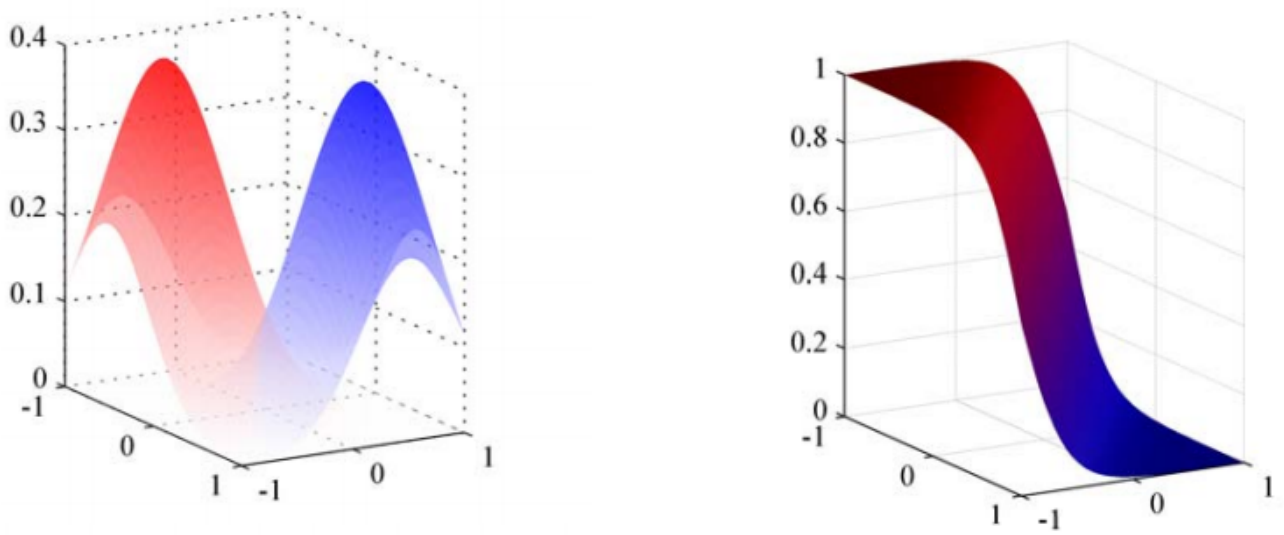


Figure 3.4.1. The left-hand plot shows the class-conditional densities for two classes, denoted red and blue. On the right is the corresponding posterior probability for class 1, which is given by a logistic sigmoid. The surface in the right-hand plot is colored using a proportion of red ink given by the class 1 conditional probability and a proportion of blue ink given by the class 2 conditional probability.

Learning: The Maximum Likelihood Principle

Given a training data set $\{(\boldsymbol{x}_1, t_1), \ldots, (\boldsymbol{x}_N, t_N)\}$ consisting of N input data points annotated with their correct class labels, we would like to learn the parameters of our model. For our generative model, the parameters are (i) the mean vector and the common covariance matrix Σ of the class-conditional densities, and (ii) the prior class probability $p(\mathcal{C}_1) = \varphi$ so that $p(\mathcal{C}_2) = 1 - \varphi$. We denote the collection of all parameters by $\boldsymbol{\theta} := (\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \boldsymbol{\Sigma}, \varphi)$.

The Likelihood. We learn the model parameters by maximising the likelihood of the training data. For a data point $\mathbf{x}_n$ from class $\mathcal{C}_1$, we have $t_n = 1$ hence

$$p(\mathbf{x}_n, \mathcal{C}_1) = p(\mathcal{C}_1)p(\boldsymbol{x}|\mathcal{C}_1) = \varphi\mathcal{N}(x_n|\boldsymbol{\mu}_1, \boldsymbol{\Sigma}).$$

where $\mathcal{N}$ denotes the Gaussian distribution. Similarly for $\mathcal{C}_2$, we have $t_n = 0$ hence

$$p(\mathbf{x}_n, \mathcal{C}_2) = p(\mathcal{C}_2)p(\boldsymbol{x}|\mathcal{C}_2) = (1-\varphi)\mathcal{N}(x_n|\boldsymbol{\mu}_2, \boldsymbol{\Sigma}).$$

Thus the likelihood function for all of the data points in the training set is:

$$\prod_{n=1}^{N} [\varphi\mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_1, \boldsymbol{\Sigma})]^{t_n} [(1-\varphi)\mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_2, \boldsymbol{\Sigma})]^{1-t_n}$$

As usual we work with the log-likelihood function since its optimisation is more convenient:

$$\mathcal{L}(\varphi, \boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \boldsymbol{\Sigma}) := \sum_{n=1}^{N} t_n \ln[\varphi\mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_1, \boldsymbol{\Sigma})] + (1-t_n)\ln[(1-\varphi)\mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_2, \boldsymbol{\Sigma})]$$

Maximising the Log-Likelihood. In order to maximise the log-likelihood $\mathcal{L}(\varphi, \boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \boldsymbol{\Sigma})$, we need to set its partial derivative with respect to each parameter to zero.

For example, if we take the terms from log likelihood that depend on φ and set its derivative with respect to φ to zero, we will obtain

$$\begin{aligned}\frac{\partial}{\partial \varphi}\mathcal{L}(\varphi, \boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \boldsymbol{\Sigma}) &= \sum_{n=1}^{N} \frac{t_n}{\varphi} - \frac{1-t_n}{1-\varphi} = 0 \\ &\Rightarrow \varphi = \frac{N_1}{N_1 + N_2}\end{aligned}$$

where N_k denotes the total number of data points in class $\mathcal{C}_k$. So the optimal value for φ is the fraction of points in the class $\mathcal{C}_1$, which is very intuitive.

Now consider the maximization with respect to μ_1. By setting the derivative to zero, we obtain:

$$\frac{\partial}{\partial \boldsymbol{\mu}_1}\mathcal{L}(\varphi, \boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \boldsymbol{\Sigma}) = 0 \Rightarrow \boldsymbol{\mu}_1 = \frac{1}{N_1}\sum_{n=1}^{N} t_n \boldsymbol{x}_n$$

which is simply the mean of all training data that belong to $\mathcal{C}_1$. Repeating the same steps for $\boldsymbol{\mu}_2$, we infer that its optimum value is the mean of all input vectors from $\mathcal{C}_2$:

$$\frac{\partial}{\partial \boldsymbol{\mu}_2}\mathcal{L}(\varphi, \boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \boldsymbol{\Sigma}) = 0 \Rightarrow \boldsymbol{\mu}_2 = \frac{1}{N_2}\sum_{n=1}^{N} (1-t_n) \boldsymbol{x}_n$$

Taking similar steps for the covariance matrix, we obtain:

$$\boldsymbol{\Sigma} = \frac{N_1}{N}\mathbf{S}_1 + \frac{N_2}{N}\mathbf{S}_2$$

$$\mathbf{S}_1 = \frac{1}{N_1}\sum_{n \in \mathcal{C}_1} (\mathbf{x}_n - \mu_1)(\mathbf{x}_n - \mu_1)^T$$

$$\mathbf{S}_2 = \frac{1}{N_2}\sum_{n \in \mathcal{C}_2} (\mathbf{x}_n - \mu_2)(\mathbf{x}_n - \mu_2)^T$$

From above expressions, we observe that the shared covariance matrix is nothing but a weighted average of the class-specific covariance matrices $\mathbf{S}_1$ and $\mathbf{S}_2$.

5
Activity 3.2 Bayesian Classifier

Probabilistic Generative Models

In this activity we implement a Bayesian Classifier for binary classification problems. This activity helps you to complete Assignment 2.

Please download this JuPyteR file (Activity.3.2 (https://www.alexandriarepository.org/wp-content/uploads/20160901074943/Activity.3.2.ipynb) right click and then choose "Save link as").

For detailed discussion about setting your programming environment up, please refer to Appendix B: Setting up Your Programming Environme (https://www.alexandriarepository.org/?post_type=module&p=97654)

6
Probabilistic Discriminative Models

Logistic Regression

Probabilistic discriminative models use generalised linear models to directly get the posterior probability of classes, and aim to learn the parameters by maximum likelihood. Specifically, we explore *logistic regression* which is a very popular probabilistic discriminative model for classification. We then present an iterative algorithm, based on the gradient descent algorithm, to learn the parameters of the model.

The Model. From the previous chapter, recall that in a binary classification setting and under some assumptions, the posterior probability of class $\mathcal{C}_1$ can be written as a logistic sigmoid acting on a linear function of the input vector

$$p(\mathcal{C}_1|\boldsymbol{x}) = \frac{1}{1+\exp -\boldsymbol{w}\cdot\boldsymbol{x}}$$

In the probabilistic discriminative models, we directly work with the above functional form for the posterior.

It is instructive to compare the number of parameters of a generative model and its corresponding discriminative model. Suppose the dimension of $\mathbf{x}$ is denoted by D. Then, in the generative model we need $D(D+1)/2$ parameters for the (shared) covariance matrix, $2D$ parameters for the two means, and 1 parameter for the class prior $p(\mathcal{C}_1)$; this gives a total of $D(D+5)/2+1$ parameters which grows quadratically with D. In contrast there are only D parameters in the discriminative model (or $D+1$ if we add a bias term as well), which makes the number of parameters to be linear in D. Hence, the discriminative model has less number of parameters compared to its corresponding generative model, which makes it less prone to overfitting and more generalisable.

The Likelihood. To learn the parameters of the model, we need to construct the likelihood function and then maximise it. For a data set $\{(\boldsymbol{x}_1, t_1), \ldots, (\boldsymbol{x}_N, t_N)\}$ of N input data point $\boldsymbol{x}$ and their labels $t \in \{0, 1\}$, the log-likelihood function can be written as:

$$\mathcal{L}(\boldsymbol{w}) := \log \textstyle\prod_{n=1}^{N} y_n^{t_n} (1-y_n)^{1-t_n}$$

where for convenience we have defined $y(\boldsymbol{x}) := p(\mathcal{C}_1|\boldsymbol{x}) = \sigma(\boldsymbol{w}\cdot\boldsymbol{x})$.

Optimising the Likelihood. We set the gradient to zero to find the optimal parameters:

$$\frac{\partial}{\partial \boldsymbol{w}}\mathcal{L}(\boldsymbol{w}) = 0 \Rightarrow \textstyle\sum_{n=1}^{N} \big(t_n - \sigma(\boldsymbol{w}\cdot\boldsymbol{x})\big)\boldsymbol{x} = \mathbf{0}$$

However, there is no analytical solution to this *non-linear* system of equation (note that the non-linearity

is introduced by the sigmoid function). Therefore, we resort to iterative algorithms to find the optimal solution.

Stochastic Gradient Descent (SGD). We have already seen SGD in Module 2 (https://www.alexandriarepository.org/reader-preview/84662/91471) for learning the parameters of regression models. Here we apply SGD to the training objective of the logistic regression to learn the parameters. The *error* function to minimise is the *negative* log-likelihood.

The algorithm works as follows. Initially, the parameters are set to zero. Then after processing each data point $(\boldsymbol{x}_n, t_n)$, the parameter vector is updated via:

$$\boldsymbol{w}^{(\tau+1)} := \boldsymbol{w}^{(\tau)} - \eta^{(\tau)} (y_n - t_n) \boldsymbol{x}_n$$

where $(y_n - t_n)\boldsymbol{x}_n$ is the gradient of the error function, τ is the iteration number, and $\eta^{(\tau)}$ is the iteration-specific learning rate. The algorithm stops when a stopping condition is met, for example the difference between the values of the objective function in two consecutive iterations is less than a threshold $|\mathcal{L}(\boldsymbol{w}^{(\tau+1)}) - \mathcal{L}(\boldsymbol{w}^{(\tau)})| < \epsilon$.

7
Activity 3.3 Logistic Regression

Probabilistic Discriminative Model

In this activity we implement Logistic Regression which is a two-class classifier. This activity helps you to complete Assignment 2.

Please download this JuPyteR file (right click Activity.3.3 (https://www.alexandriarepository.org/wp-content/uploads/20160901074730/Activity.3.3.ipynb) and then choose "Save link as").

For detailed discussion about setting your programming environment up, please refer to Appendix B: Setting up Your Programming Environme (https://www.alexandriarepository.org/?post_type=module&p=97654)